

Optimización del rendimiento de un hilo simple de ejecución en procesadores de propósito general

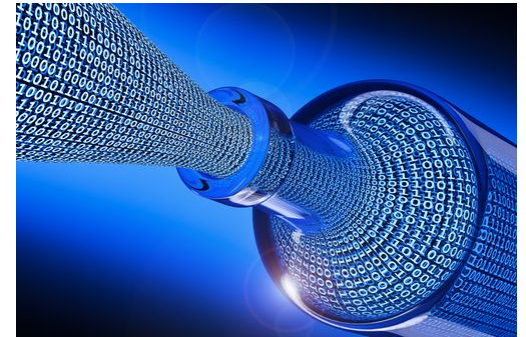
Agenda

1. Limitaciones al rendimiento computacional.
2. Roles del compilador y del programador, y sintonización de la compilación.
3. Técnicas generales de optimización de código.
4. Instrucciones vectoriales.
5. Colaboración con el compilador de C.

Limitaciones al rendimiento computacional

Cómputo limitado por memoria (*memory bound*)

- La CPU está mucho tiempo ociosa a causa de la lentitud del subsistema de memoria en entregar datos para procesar.



- Si el acceso a los datos en memoria fuera más rápido (mayor ancho de banda), la ejecución también sería más rápida.

Cómputo limitado por CPU (*CPU bound*)

- La CPU computa a su máxima capacidad y nunca queda ociosa por falta de datos para computar.

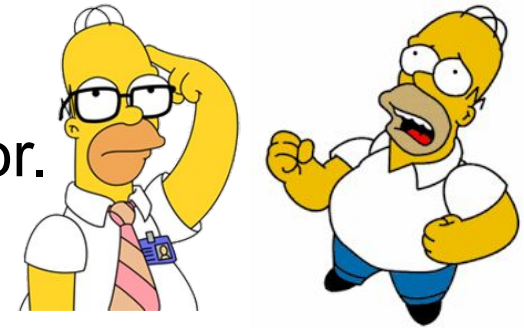


- Si la CPU fuera más rápida, la ejecución también sería más rápida.

Roles del compilador y del programador, y sintonización de la compilación

Roles del compilador y del programador

- Los compiladores pueden ser sorprendentemente inteligentes y estúpidos al mismo tiempo. La incapacidad de un compilador en optimizar un código puede deberse a:
 - La situación no era tan trivial para el compilador.
 - El programa no colaboró con el compilador.
- Recomendaciones al programador:
 - Aplicar técnicas de optimización de código de forma manual para no depender de la capacidad del compilador.
 - Escribir programas que colaboren con la optimización automática del compilador.



Sintonización de la compilación

Microarquitectura del procesador

- Para que el compilador genere un código de máquina eficiente es importante indicar la microarquitectura del procesador.
 - Esto proporciona información sobre el hardware (tamaños de cachés, líneas de cachés, ISAs implementados, etc.), útil para que el compilador sintonice el código de máquina.
- Ejemplo en GCC:
 - march=nombre de la microarquitectura

Sintonización de la compilación

Niveles de optimizaciones

- Cada compilador define sus propios niveles que se describen en su documentación. En GCC se puede elegir el nivel con la opción **-Onivel**.
- Todos los compiladores se abstienen de la mayoría de las optimizaciones en el nivel cero, por lo tanto, es la opción correcta para **depuración** del código. Por defecto, gcc utiliza el nivel **-O0**.
- Habilitación/deshabilitación de una técnica de optimización específica. Ejemplo en GCC: `-funroll-loops`

Técnicas generales de optimización de código

Hacer menos trabajo y más liviano

Si hay dos formas de resolver algo, es mejor aquella que reduce la cantidad y complejidad de las instrucciones del procesador utilizadas.

Reducción del almacenamiento para datos

Utilizar el menor espacio posible para almacenar los elementos de datos en memoria permite:

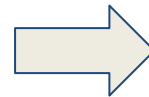
1. Reducir el problema del cómputo limitado por memoria, dado que más elementos de datos entrarán en memoria caché.
2. Que problemas más grandes entren en la memoria principal.
3. Aumentar el paralelismo de datos al utilizar procesamiento vectorial.
4. Reducir el consumo de energía.
5. Acelerar la E/S en almacenamiento secundario si éste se requiere.

Importante: verificar que el procesador tenga instrucciones que permitan operar eficientemente sobre los tamaños de datos elegidos.

Extracción de subexpresiones comunes en repetitivas

La extracción de subexpresiones comunes en construcciones repetitivas evita realizar cálculos repetidos.

```
for (i = 0; i < n; i++)  
    A[i] = A[i] + s + r * sin(x);
```



```
tmp = s + r * sin(x);  
for (i = 0; i < n; i++)  
    A[i] = A[i] + tmp;
```

El compilador podría no aplicar automáticamente esta optimización cuando los cálculos se realizan con elementos de coma flotante.

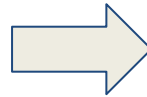
Código en línea

- El código en línea reduce la sobrecarga por llamadas a funciones/subrutinas ya que no se requiere implementar lo estipulado por las convenciones de llamadas.
- Es útil cuando la sobrecarga es significativa en relación a la complejidad de la función/subrutina.

Desenrollado de bucles

- Consiste en reducir la cantidad de iteraciones de los bucles y aumentar la carga de trabajo por iteración.

```
for (i = 0; i < 100; i++)  
    x[i] = x[i] + s;
```



```
for (i = 0; i < 100; i = i + 2) {  
    x[i] = x[i] + s;  
    x[i+1] = x[i+1] + s;  
}
```

- Ventajas:
 - Menor cantidad de instrucciones ejecutadas.
 - Menor cantidad de dependencias de control.
 - Puede permitir un posterior procesamiento vectorial.
- Desventajas:
 - Aumenta el tamaño del código.
 - Aumenta la cantidad de registros de CPU requeridos por iteración.

Evitación de saltos condicionales

- Se basa en disminuir la cantidad de saltos condicionales para reducir las dependencias de control.
- Los saltos condicionales que están dentro de los bucles podrían llegar a impedir el procesamiento vectorial.

Optimizaciones de memoria

- **Código amigable con la caché:** se busca incrementar los aciertos de caché para aumentar el rendimiento. Para esto, el acceso a datos debe considerar los principios de localidad espacial y temporal de la memoria caché.
 - **Optimización de bucles anidados mediante particionado de bucles (loop blocking/tiling).** Cuando hay acceso repetido a estructuras de datos que superan el tamaño de la caché (L1, L2 o L3), siempre que sea posible, realizar cómputo sobre una región de la estructura (que entre en caché) por vez. Mejora la localidad temporal.

Optimizaciones de memoria

- **Alineación de datos en memoria:**

- Si un elemento de dato se alinea en memoria de tal manera que luego no traspase las fronteras de una línea de caché, su acceso sólo afectará a una única línea de caché en vez de a dos.
- Algunas instrucciones de alto rendimiento requieren que los datos tengan una determinada alineación en memoria.

Ejemplo de alineación de memoria en el lenguaje C:

- Memoria estática: ejemplo de variable `x` alineada a 16 bytes.

```
int x __attribute__((aligned (16)));
```

- Memoria dinámica: utilizar `posix_memalign` en lugar de `malloc`.

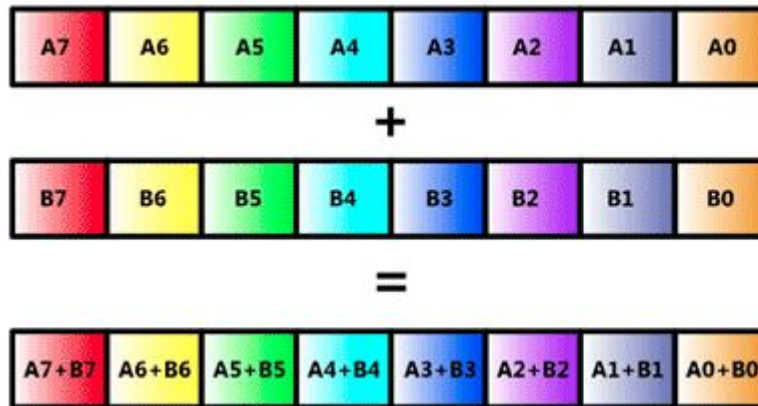
```
int posix_memalign(void **memptr, size_t alignment, size_t size);
```

Instrucciones vectoriales

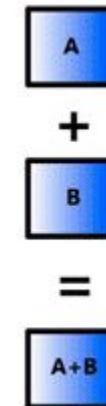
Instrucciones vectoriales

Instrucciones que se aplican sobre vectores, siguiendo el modelo SIMD de la taxonomía de Flynn. Aumenta el rendimiento respecto a una solución con instrucciones escalares.

Instrucción vectorial



Instrucción escalar



Vectorización automática

- El compilador aplica vectorización automática para **casos triviales**.
- Es posible indicar al compilador que habilite la opción de optimización de vectorización automática. En GCC:

-ftree-vectorize

Puede estar incluida en algún nivel predefinido de optimización:

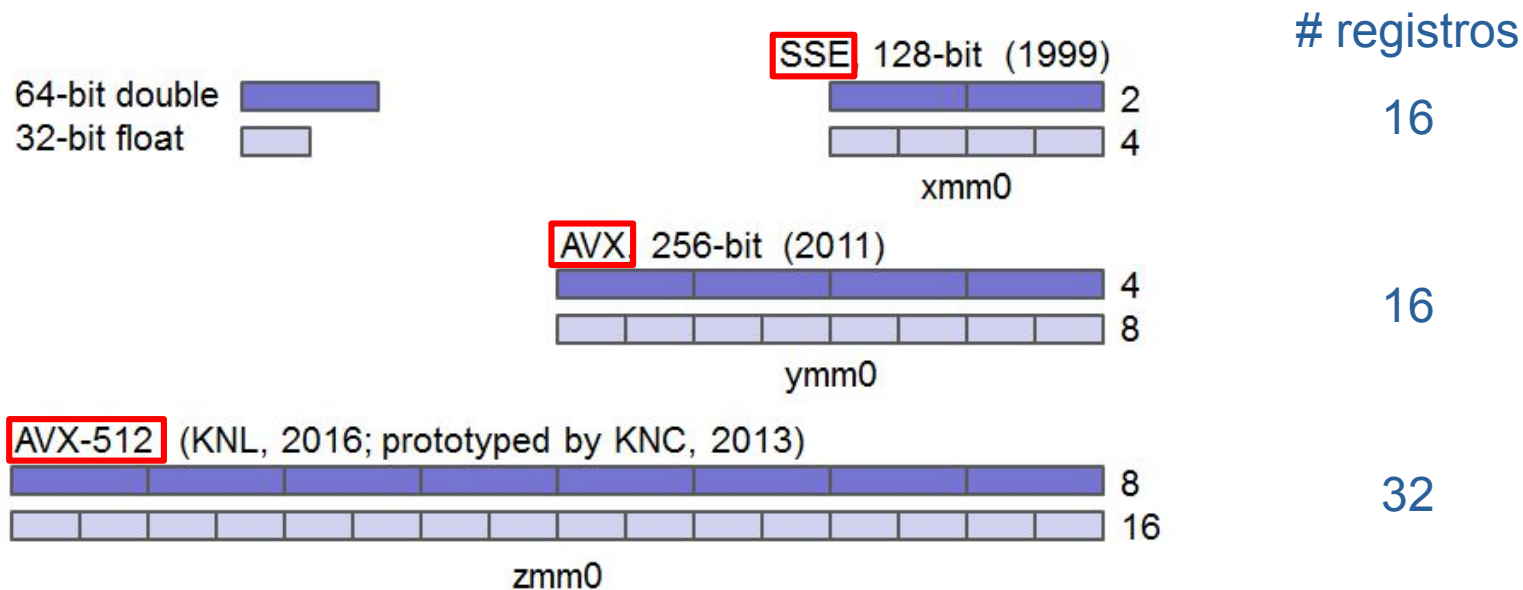
en GCC, se encuentra en el nivel 3, opción **-O3**

Funciones intrínsecas

- Cuando el compilador no es capaz de vectorizar un código, el programador deberá diseñar e implementar un algoritmo vectorizado.
- Si el lenguaje de alto nivel no dispone de construcciones que permitan utilizar las instrucciones vectoriales, se pueden utilizar las **funciones intrínsecas** que implementan los compiladores. Normalmente, estas funciones tienen una traducción uno a uno con las instrucciones de lenguaje ensamblador.

Juegos de instrucciones vectoriales

- Las instrucciones vectoriales trabajan sobre **registros** más largos que las destinadas a operaciones escalares. Cuanto más pequeños son los elementos de datos, más entrarán en los registros, y entonces más operaciones se realizarán por instrucción.
- Hay diversos juegos de instrucciones vectoriales, por ejemplo:



- Los juegos de instrucciones no solo varían en el tamaño de los registros sino también en los tipos de datos y operaciones soportadas.

Arquitectura de juegos de instrucciones vectoriales

- La arquitectura de los juegos de instrucciones vectoriales es del tipo **carga-almacenamiento**. Procesamiento típico:
 1. Se leen los datos de la memoria y se cargan en registros.
 2. Se procesan los datos de los registros.
 3. Se almacenan los datos de los registros en la memoria.
- Las instrucciones de carga-almacenamiento suelen estar en dos versiones, una de alto rendimiento que trabaja con **datos alineados en memoria** y otra de bajo rendimiento que trabaja con datos no alineados en memoria.
 - La función `posix_memalign` permite solicitar al sistema operativo la asignación de memoria dinámica con el requisito de que la dirección de memoria inicial del bloque asignado sea múltiplo de un cierto valor de alineación indicado.

Arquitectura de juegos de instrucciones vectoriales

Ejemplo: AVX - 256 bits

Declaración de vectores (registros)

`__m256` es un vector de 8 floats

`__m256d` es un vector de 4 doubles

`__m256i` es un vector de enteros (sin distinción del tamaño de los enteros)

Nombres de las funciones intrínsecas

- Normalmente, los nombres indican el tamaño de los registros, operación y tipo de los elementos de datos:

`__mm<tamaño en bits>_<operación>_<tipo de dato>`

- Algunos tipos de datos son:

Tipo de dato	Descripción
epi8, epi16, epi32	entero de 8, 16 o 32 bits con signo
eput8, eput16, eput32	entero de 8, 16 o 32 bits sin signo
ps / pd	flotante de simple/doble precisión
si256	256 bits de datos enteros

Manual de referencia de funciones vectoriales intrínsecas

<https://software.intel.com/sites/landingpage/IntrinsicsGuide/>



The Intel Intrinsics Guide is an interactive reference tool for Intel intrinsic instructions, which are C style functions that provide access to many Intel instructions - including Intel® SSE, AVX, AVX-512, and more - without the need to write assembly code.

Technologies

- ☐ MMX
- ☐ SSE
- ☐ SSE2
- ☐ SSE3
- ☐ SSSE3
- ☐ SSE4.1
- ☐ SSE4.2
- ☒ AVX
- ☒ AVX2
- ☐ FMA
- ☐ AVX-512
- ☐ KNC
- ☐ AMX
- ☐ SVML
- ☐ Other

Categories

- ☐ Application-Targeted
 - ☒ Arithmetic
 - ☐ Bit Manipulation
 - ☐ Cast
 - ☐ Compare
 - ☐ Convert
 - ☐ Cryptography
 - ☐ Elementary Math
- ## Functions
- ☐ General Support
 - ☐ Load
 - ☐ Logical
 - ☐ Mask
 - ☐ Miscellaneous
 - ☐ Move

_mm_search

<code>__m256i _mm256_add_epi16 (__m256i a, __m256i b)</code>	vpaddw
<code>__m256i _mm256_add_epi32 (__m256i a, __m256i b)</code>	vpaddq
<code>__m256i _mm256_add_epi64 (__m256i a, __m256i b)</code>	vpaddq
<code>__m256i _mm256_add_epi8 (__m256i a, __m256i b)</code>	vpaddb
<code>__m256d _mm256_add_pd (__m256d a, __m256d b)</code>	vaddpd
<code>__m256 _mm256_add_ps (__m256 a, __m256 b)</code>	vaddps

Synopsis

```
__m256 _mm256_add_ps (__m256 a, __m256 b)
#include <immintrin.h>
Instruction: vaddps ymm, ymm, ymm
CPUID Flags: AVX
```

Encabezado a incluir en el programa

Instrucción en ensamblador

Si el comando `lscpu` no incluye el flag es porque ese ISA no está soportado por la CPU.

Description

Add packed single-precision (32-bit) floating-point elements in `a` and `b`, and store the results in `dst`.

Operation

```
FOR j := 0 to 7
    i := j*32
    dst[i+31:i] := a[i+31:i] + b[i+31:i]
ENDFOR
dst[MAX:256] := 0
```

Descripción de la funcionalidad

Performance

Architecture	Latency	Throughput (CPI)
Icelake	4	0.5
Skylake	4	0.5
Broadwell	3	1
Haswell	3	1
Ivy Bridge	3	1

Detalles de rendimiento en microarquitecturas de Intel

Ejemplo utilizando SSE2

```
#include <stdio.h>
#include <stdlib.h>
#include <emmintrin.h>
```

```
int main() {
    int i;
    char *a, *b, *c;
    __m128i v1, v2;
```

```
    if(posix_memalign((void**)&a, 16, 16) != 0)
        return 1;

    if(posix_memalign((void**)&b, 16, 16) != 0)
        return 1;

    if(posix_memalign((void**)&c, 16, 16) != 0)
        return 1;
```

```
    for (i = 0; i < 16; i++) {
        a[i] = i;
        b[i] = 1;
    }
```

```
v1 = _mm_load_si128((__m128i*)a);
v2 = _mm_load_si128((__m128i*)b);
```

```
v1 = _mm_adds_epi8(v1,v2);
```

```
_mm_store_si128((__m128i*)c, v1);
```

```
for (i = 0; i < 16; i++)
    printf("%d ", c[i]);
printf("\n");
```

```
return 0;
```

```
}
```

Compilación

Es necesario indicar al compilador cuáles juegos de instrucciones vectoriales están disponibles para que nuestro programa utilice. Dos posibilidades:

- Indicar la microarquitectura del procesador:

`-march=nombre de la microarquitectura`

Esto permite al compilador conocer cuáles son todos los conjuntos de instrucciones que soporta el procesador.

- Indicar el o los juegos de instrucciones específicos:

Ejemplo en GCC: `-msse2 -mavx`

Colaboración con el compilador de C

Declaración de literales float

Al operar variables float con valores constantes, utilizar sufijo f/F (ej. 3.14f).

- Si no se hace:
 - Aumentará el espacio de memoria requerido.
 - Se utilizarán innecesariamente instrucciones más complejas y, si el resultado debe ser de tipo *float*, se agregará la conversión a ese tipo de dato.
 - Una vectorización automática del código utilizará instrucciones vectoriales para tipos *double* en vez de *float*, reduciendo la cantidad de operaciones simultáneas a la mitad.

Uso de arreglos y punteros

Utilizar notación de arreglos en vez de notación de punteros cuando se trabaja con arreglos.

El compilador a veces asume el *aliasing* (un dato puede accederse a través de dos punteros diferentes) ante la presencia de punteros, que limitan las oportunidades de optimización. La notación de arreglos hace que el compilador sepa que no hay *aliasing*.

Declaración estática de funciones locales

Utilizar el calificador *static* en funciones que solo son requeridas por otras funciones del mismo archivo de programa.

Esto permite al compilador utilizar la técnica de código en línea con estas funciones.

Uso del calificador const

Si el dato nunca se modifica, declararlo como constante.

- Un compilador no requiere asignar almacenamiento en memoria principal para un objeto que se declara constante, si nunca se requiere su dirección.

Orden de operandos booleanos

Si se tienen operadores `&&` (*and*), poner a la izquierda los operandos que más probabilidad tienen de ser evaluados como falsos.

Si se tienen operadores `||` (*or*), poner a la izquierda los operandos que más probabilidad tienen de ser evaluados como verdaderos.

- Esta estrategia permite acelerar las evaluaciones de expresiones lógicas al evitar la evaluación de todos los operandos que la componen.

Uso del calificador register

Recomendar al compilador almacenar datos (con altas tasas de acceso) en registros en vez de en memoria RAM.

- Esta estrategia permite que los datos muy utilizados se accedan rápidamente.

Ejemplo:

```
register int i;
```

Bibliografía

- Libro de Grama
 - 2.2 Limitations of Memory System Performance
 - 2.2.1 Improving Effective Memory Latency Using Caches
 - 2.2.2 Impact of Memory Bandwidth
- Libro de Hager:
 - 2.2 Common sense optimizations
 - 2.3 Simple measures, large impact
 - 2.4 The role of compilers
- Software Optimization Guide for AMD Family 15h Processors.
http://support.amd.com/TechDocs/47414_15h_sw_opt_guide.pdf
- Intel 64 and IA-32 Architectures Optimization Reference Manual
<http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf>
- Intel Intrinsics Guide
<https://software.intel.com/sites/landingpage/IntrinsicsGuide/>
- Otros manuales sobre optimizaciones:
<https://www.agner.org/optimize/#manuals>